



CanlıFal — Backend Geliştirici Dokümantasyonu

Base URL: `https://canlifal.com`

Sürüm: 1.0.0 · **Son Güncelleme:** 23 Temmuz 2026

Kapsam: Web ve Flutter mobil uygulaması **AYNI** backend'i ve **AYNI** PostgreSQL veritabanını kullanır. Bu doküman, Flutter uygulamasının web ile %100 aynı çalışması için gereken eksiksiz backend referansıdır.



Teslim Edilen Dosyalar

Bu dokümantasyon paketi (`backend-docs/` klasörü) aşağıdaki dosyalardan oluşur:

Dosya	Açıklama
<code>CANLI-FAL_BACKEND_GELISTIRICI_DOKUMANTASYONU.md</code> (+ <code>.pdf/.docx</code>)	Bu ana kılavuz
<code>backend-docs/openapi.json</code>	OpenAPI 3.0 spesifikasyonu — 468 yol, 731 operasyon. Swagger UI / Editor'e import edilebilir
<code>backend-docs/postman_collection.json</code>	Postman Collection v2.1 — 160 klasör, 731 istek. Doğrudan import edilir
<code>backend-docs/ENDPOINTS.md</code>	Kategorize edilmiş tüm endpoint listesi (auth + rate limit + body alanları)
<code>backend-docs/schema.prisma</code>	Tam Prisma şeması (193 model)
<code>backend-docs/database_schema.sql</code>	Tam SQL DDL (193 tablo, 149 FK, 527 index) — projenin tek migration kaynağı
<code>backend-docs/DATABASE_REFERENCE.md</code>	Tüm modeller, alanlar, ilişkiler ve indexler (insan-okunur)
<code>backend-docs/endpoints_index.json</code>	Makine-okunur ham endpoint listesi

Not — Migration dosyaları: Proje veritabanı senkronizasyonu için `prisma db push` yaklaşımını kullanır; bu nedenle klasik `prisma/migrations/*` geçmiş dosyaları **yoktur**. Bunun yerine `database_schema.sql`, şemanın tamamını sıfırdan kuran eksiksiz DDL'i (tek migration olarak) içerir. Yeni bir ortam kurarken bu SQL doğrudan çalıştırılabilir veya `prisma db push` kullanılabilir.

İçindekiler

1. Genel Mimari
2. Kimlik Doğrulama — JWT & Refresh Token Akışı
3. Yetkilendirme (Authorization) Kuralları
4. Yanıt Zarfı (Response Envelope) Konvansiyonu
5. Hata Kodları
6. Validation (Doğrulama) Kuralları
7. Rate Limit Kuralları
8. Endpoint Kategorileri (Genel Bakış)
9. Auth Endpoint'leri — Tam Örnekler
10. Veritabanı Modelleri & İlişkiler
11. Swagger / Postman Kullanımı

1. Genel Mimari

- **Tek backend, iki istemci:** Web (tarayıcı) ve Flutter mobil uygulaması aynı REST API'yi tüketir. Endpoint'lerin büyük çoğunluğu **dual-auth** (çift kimlik doğrulama) destekler: aynı uç hem web oturum çerezi hem de mobil JWT Bearer token ile çalışır.
- **Toplam kapsam:** 468 benzersiz endpoint yolu, 731 HTTP handler, 160 fonksiyonel kategori.
- **Veritabanı:** PostgreSQL, Prisma ORM ile yönetilir. 193 model / tablo.
- **Auth dağılımı:** 393 handler dual-auth · 238 handler web oturumu · 100 handler public. 301 handler yönetici (admin) korumalıdır.
- **Gerçek zamanlı:** SSE (Server-Sent Events) akışları oda/yayın/bildirim olayları için kullanılır (/ stream uçları). WebRTC sinyalleşmesi `signal` uçları üzerinden yapılır.

2. Kimlik Doğrulama — JWT & Refresh Token Akışı

Mobil uygulama **JWT tabanlı** kimlik doğrulama kullanır. Web tarafı NextAuth oturum çerezi kullanır. Dual-auth uçları her ikisini de kabul eder.

2.1 Token Yapısı

Token	Süre	Amaç
accessToken	7 gün	Her API isteğinde <code>Authorization: Bearer <accessToken></code> header'ında gönderilir
refreshToken	30 gün	accessToken süresi dolduğunda yeni token çifti almak için kullanılır

JWT payload alanları: `userId`, `email`, `role`, `type` (`access` | `refresh`). İmzalama algoritması: HS256 (sunucu gizli anahtarı ile).


```
{
  "email": "yeni@ornek.com", "password": "sifre123", "name": "Ayşe",
  "username": "ayse", "birthDate": "1995-05-20", "birthTime": "14:30",
  "referralCode": "ABC123", "preferredLanguage": "tr"
}
```

Zorunlu: email, password, name, username, birthDate, birthTime . Opsiyonel: referralCode , preferredLanguage .

Yanıt (200): accessToken + refreshToken + user .

Hatalar: 400 zorunlu alan eksik / geçersiz e-posta / e-posta veya kullanıcı adı zaten kayıtlı · 429 rate limit.

2.6 Sosyal Giriş

Endpoint	Açıklama
POST /api/auth/mobile-google	Google ID token ile giriş/kayıt
POST /api/auth/mobile-apple	Apple kimlik token'ı ile giriş/kayıt
POST /api/auth/mobile-tiktok	TikTok OAuth ile giriş/kayıt

Tümü başarıda accessToken + refreshToken + user döner.

2.7 Diğer Auth Uçları

Endpoint	Açıklama
POST /api/auth/forgot-password	Şifre sıfırlama e-postası / kodu gönderir
POST /api/auth/reset-password	Kod/token ile yeni şifre belirler
POST /api/auth/change-password	Oturumlu kullanıcı şifre değiştirir
POST /api/auth/logout	Oturumu/token'ı sonlandırır
POST /api/auth/verify-device	Cihaz doğrulama
POST /api/auth/reclaim-device	Cihaz geri alma

3. Yetkilendirme (Authorization) Kuralları

3.1 Kimlik Doğrulama Katmanı

Her korumalı uç authenticateRequest(req) yardımcısını kullanır. Bu fonksiyon:

- Önce** Authorization: Bearer <token> header'ını kontrol eder (mobil). Token geçerli ve type === 'access' ise kullanıcıyı döner (performans için cache'lenir).
- Aksi halde** NextAuth web oturumunu kontrol eder (getSession).

3. İkisi de yoksa `null` döner → uç `401 Unauthorized` verir.

```
// Dual-auth kalıbı (her korumalı uçta):
const user = await authenticateRequest(req)
if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
```

3.2 Rol Tabanlı Erişim

`User.role` alanı String'dir. Tanımlı roller:

Rol	Açıklama
user	Standart kullanıcı (varsayılan)
admin	Tam yönetici erişimi
yonetici	Yönetici (admin ile eşdeğer panel erişimi)
moderator	Moderasyon yetkileri (sınırlı)
finans	Finans/ödeme yönetimi (sınırlı)

Admin korumalı uçlar (301 handler) genellikle şu kontrolü içerir:

```
if (!['admin', 'yonetici'].includes((session.user as any).role)) {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 })
}
```

Bazı panel uçları ek olarak `moderator` / `finans` rollerine de izin verir (örn. ödeme yöntemleri sayfası).

3.3 Sahiplik & Finans Muafiyeti

- **Sahiplik:** Kullanıcıya özel veri uçları (siparişler, mesajlar, oturumlar) `userId` 'yi **oturumdan/token'dan** türetir, URL parametresinden değil. Başka kullanıcının verisine erişim `403` döner.
- **Finanstan muaf hesaplar:** `isExcludedFromFinance(userId)` yardımcısı ile işaretlenen personel/yönetici hesapları jeton harcarken/kazanırken bakiye değişmez (test/moderasyon amaçlı sınırsız bakiye).

4. Yanıt Zarfı (Response Envelope) Konvansiyonu

İki kalıp kullanılır (uç bazında değişir):

A) Zarf'lı (yeni/mobil uçların çoğu):

```
{ "success": true, "data": { /* ... */ } }
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Oturum açmanız gerekiyor" } }
```

B) Düz (bazı web uçları):

```
{ "streams": [...], "pagination": {...} }
{ "error": "Yetersiz jeton bakiyesi" }
```

Flutter önerisi: İstemci tarafında her iki kalıbı da tolere eden bir parser yazın: önce `success` alanını kontrol edin, yoksa gövdeyi doğrudan veri olarak kabul edin. HTTP durum kodu her zaman doğru sinyaldir (2xx = başarı, 4xx/5xx = hata).

5. Hata Kodları

5.1 HTTP Durum Kodları

Kod	Anlam
200	Başarılı
400	Geçersiz istek / eksik alan / doğrulama hatası
401	Kimlik doğrulama gerekli veya token geçersiz/süresi dolmuş
403	Yetki yok (yanlış rol / sahiplik ihlali)
404	Kaynak bulunamadı
429	Rate limit aşıldı (Retry-After dikkate alın)
500	Sunucu hatası
503	Servis hazır değil (örn. yapılandırılmamış özellik)

5.2 Uygulama Hata Kodları (`error.code` / `error`)

En sık kullanılan makine-okunur hata kodları:

Kod	Açıklama
INTERNAL_ERROR	Beklenmeyen sunucu hatası
UNAUTHORIZED	Kimlik doğrulama başarısız
FORBIDDEN	Yetkisiz erişim
NOT_FOUND	Genel kaynak bulunamadı
INVALID_PARAMS / MISSING_PARAMS	Geçersiz veya eksik parametre
USER_NOT_FOUND	Kullanıcı bulunamadı
ROOM_NOT_FOUND / ROOM_INACTIVE	Oda bulunamadı / aktif değil
STREAM_NOT_FOUND / STREAM_ENDED	Yayın bulunamadı / sona erdi
SEAT_TAKEN	Koltuk dolu
INSUFFICIENT_BALANCE	Yetersiz jeton bakiyesi
SELF_GIFT	Kendine hediye gönderilemez
RECIPIENT_NOT_FOUND / NO_RECIPIENT	Alıcı bulunamadı
NOT_A_TELLER / NOT_APPROVED	Falcı değil / onaylı değil
PK_NOT_FOUND / PK_EXPIRED / PK_NOT_ACTIVE / PK_EXISTS	PK Battle durumları
TRTC_NOT_CONFIGURED	Canlı görüşme altyapısı yapılandırılmamış
COOLDOWN_ACTIVE	Bekleme süresi aktif
ACCOUNT_RESTRICTED	Hesap kısıtlı
UPLOAD_FAILED / PRESIGN_FAILED	Dosya yükleme hatası

Kullanıcıya gösterilen `message` alanları Türkçedir. İstemci mantığı için `code` alanını (veya HTTP durumunu) kullanın, mesaj metnini değil.

6. Validation (Doğrulama) Kuralları

Doğrulama, her uçta el ile (imperatif) yapılır. Genel kurallar:

6.1 Kimlik / Hesap

- **E-posta:** `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` regex'i ile doğrulanır; küçük harfe çevrilip trim edilir. Benzersiz olmalıdır.

- **Kullanıcı adı (username):** Benzersiz, küçük harfe normalize edilir. Zorunludur (kayıtta).
- **Şifre:** bcrypt ile hash'lenir (salt rounds = 10). Minimum uzunluk uça kontrol edilir.
- **birthDate / birthTime:** Kayıta zorunlu. birthTime HH:mm formatında (yükselen burç hesabı için).
- **referralCode:** Opsiyonel; büyük harfe çevrilerek eşleştirilir.

6.2 Genel Girdi Kuralları

- Zorunlu alan eksikse 400 + açıklayıcı Türkçe mesaj döner.
- Sayısal alanlar (örn. hediye quantity , beğeni count) makul üst sınırlara cap'lenir (örn. beğeni count maksimum 100).
- Sayfalama: page (varsayılan 1), limit (varsayılan 30, maksimum 100).
- Mesaj/yorum içeriği boş olamaz; boşluk trim edilir.
- Parasal/jeton işlemlerinde bakiye yeterliliği işlemten önce kontrol edilir; her hareket balance-Before / balanceAfter ile loglanır.

Not: Proje zod gibi bir şema doğrulama kütüphanesi kullanmaz; doğrulama route handler'ları içinde yapılır. Bu nedenle Flutter tarafında da istemci-tarafı doğrulama (form validation) uygulamanız, sunucu hatalarını azaltır.

7. Rate Limit Kuralları

Sunucu, bellek-içi **token-bucket** rate limiter kullanır. Anahtar: kimlikli isteklerde userId , aksi halde IP (x-forwarded-for).

7.1 Tanımlı Limitler

Limitler	Limit	Kullanım
authLimiter	10 istek / 15 dakika	Giriş, kayıt, sosyal giriş, şifre sıfırlama (kimlik doğrulama uçları)
apiLimiter	60 istek / dakika	Genel API uçları
heavyLimiter	10 istek / dakika	Ağır işlemler (yoğun kaynak kullanan uçlar)

7.2 Davranış

- Limit aşılnca uç 429 döner: { "error": "Çok fazla istek. Lütfen biraz bekleyin." } .
- retryAfter (saniye) hesaplanır; istemci bu süre kadar beklemelidir.
- **Flutter önerisi:** 429 alınca üstel geri çekilme (exponential backoff) uygulayın ve kullanıcıya "biraz sonra tekrar deneyin" mesajı gösterin. Auth uçlarında agresif retry yapmayın (15 dk pencere).

Bellek-içi limiter, sunucu örneği başına çalışır (dağıtık değildir). Kritik senaryolarda istemci tarafı istek sıklığını da sınırlamak önerilir.

8. Endpoint Kategorileri (Genel Bakış)

Toplam **160 kategori / 731 handler**. Tam liste için `backend-docs/ENDPOINTS.md` , makine-okunur hali için `openapi.json` ve `postman_collection.json` .

Başlıca fonksiyonel alanlar:

- **auth/** — Giriş, kayıt, token yenileme, sosyal giriş, şifre işlemleri.
- **user/**, **users/**, **profile/** — Profil, ayarlar, bakiye, takip, engelleme, cihazlar.
- **fortune-tellers/**, **room/**, **fortune/** — Falcılar, canlı fal seansları, WebRTC sinyalleşme, mesajlaşma, yorumlar.
- **video-streams/** — Canlı yayınlar, izleyiciler, yorumlar, beğeniler, hediyeler, sinyalleşme.
- **chat/**, **live/** — Sesli sohbet odaları, koltuklar, mesajlaşma, PK Battle, çevrimiçi kullanıcılar.
- **gifts/** — Hediye kataloğu (CMS senkronizasyonu), gönderme, **Şanslı Hediye** (config/send/history).
- **credits/**, **payments/**, **credit-packages/**, **cfc/** — Jeton paketleri, ödemeler, para birimi, kazanç/çekim.
- **mobile/** — Mobil'e özel toplu (compound) uçlar: `config` , `home` , `fortune-menu` , `user-profile` .
- **horoscope/**, **dreams/**, **blog/**, **celebrities/**, **social/** — İçerik ve topluluk.
- **notifications/** — Bildirimler, push (OneSignal), okundu işaretleme.
- **admin/** — 301 handler: içerik yönetimi, kullanıcı/rol yönetimi, finans, ayarlar, istatistik, moderasyon, hediye/koleksiyon (Şanslı Hediye kademeleri dahil).

9. Auth Endpoint'leri — Tam Örnekler

Bölüm 2'de detaylandırılmıştır. Diğer tüm uçların istek/yanıt şekilleri için:

- **Swagger:** `openapi.json` → Swagger UI'da her uç için parametre, body ve güvenlik şeması görünür.
- **Postman:** `postman_collection.json` → her istek hazır; `{{baseUrl}}` , `{{accessToken}}` , `{{refreshToken}}` değişkenleri koleksiyon düzeyinde tanımlıdır.

10. Veritabanı Modelleri & İlişkiler

- **193 model / tablo.** Tam detay: `backend-docs/DATABASE_REFERENCE.md` (her modelin alanları, tipleri, nitelikleri, ilişkileri ve indexleri).
- **Tam SQL DDL:** `backend-docs/database_schema.sql` (193 CREATE TABLE, 149 FOREIGN KEY, 527 index).
- **Prisma şeması:** `backend-docs/schema.prisma` .

10.1 Enum Yaklaşımı

Bu şemada Prisma `enum` tipi kullanılmaz. Durum/rol/tip alanları `String` olarak saklanır; geçerli değerler alan açıklamalarında belirtilir. Örnekler:

- `User.role : user | admin | yönetici | moderator | finans`
- `User.membership : basic | premium | gold`
- `User.messagePrivacy : everyone | followers | nobody`

10.2 Merkezi Modeller (özet)

- **User** — Tüm ilişkilerin merkezi. Bakiyeler: `credits` , `jetonBalance` , `cfcBalance` . Rol, üyelik, profil, gizlilik ve kozmetik alanları içerir.
- **LiveFortuneTeller / LiveSession** — Falcılar ve canlı fal seansları (kredi/kazanç/komisyon mantığı).
- **VideoStream** — Canlı yayınlar (izleyici, yorum, beğeni, hediye ilişkileri).
- **ChatRoom** — Sesli sohbet odaları (koltuk, rol, mute/ban).
- **GiftType / GiftCollection** — Hediye kataloğu (CMS). `GiftType.isLucky` → Şanslı Hediye.
- **LuckyGiftTier / LuckyGiftReward** — Şanslı Hediye ödül kademeleri ve kazanç kayıtları.
- **Payment / CreditPackage / PaymentMethod** — Ödeme ve jeton satın alma.
- **PlatformSettings** — Dinamik yapılandırma (komisyon oranları, oda kapasiteleri, jeton-TL kuru vb.).

11. Swagger / Postman Kullanımı

Swagger UI ile

1. editor.swagger.io (<https://editor.swagger.io>) adresini açın.
2. **File** → **Import File** ile `backend-docs/openapi.json` dosyasını yükleyin.
3. Tüm uçları, parametreleri ve güvenlik şemalarını interaktif inceleyin.

Postman ile

1. Postman → **Import** → `backend-docs/postman_collection.json` .
2. Koleksiyon değişkenlerini ayarlayın: `baseUrl` (varsayılan `https://canlifal.com`), `accessToken` , `refreshToken` .
3. Önce **auth** → **mobile-login** isteğini çalıştırın, dönen `accessToken` 'ı koleksiyon değişkenine kopyalayın. Diğer tüm istekler bu token'ı otomatik kullanır (Bearer auth koleksiyon düzeyinde tanımlı).

Bu dokümantasyon, kaynak koddan otomatik keşifle üretilmiştir; hiçbir endpoint elle atlanmamıştır. Şema ve endpoint sayıları koddaki gerçek duruma dayanır.